



**UNIVERSIDAD
CENTROAMERICANA JOSÉ
SIMEÓN CAÑAS**

DEPARTAMENTO DE INGENIERIA Y ARQUITECTURA

BASES DE DATOS

Catedrático: Ing. James Edward Humberstone Morales

PROYECTO DE BASES DE DATOS

SISTEMA DE RESERVA DE UN HOTEL

Versión PostgreSQL: 18

Estudiantes:

Elías Samuel Rubio Hernández - 00052524

José Andrés Calles Ramírez - 00054525

Daniel Andrés Hernández Barahona - 00036225

Adrián Emanuel Escobar Aviles - 00032125

Introducción	2
Diagramas	2
Diagrama Entidad-relación (ER).....	3
Diagrama Relacional normalizado (3FN)	3
Diccionario de datos	3
Tabla: hotel.....	4
Tabla: Tipo de habitación.....	4
Tabla: Habitación	5
Tabla: Huésped.....	7
Tabla: Empleado	8
Tabla: Reserva.....	8
Tabla: Estancia	9
Tabla: Servicio	10
Tabla: Consumo servicio.....	11
Tabla: Factura.....	12
Consultas SQL en el Sistema de reservas	14
Consulta 1: Habitaciones disponibles en un rango de fechas.....	14
Consulta 2: Huéspedes con mayor gasto histórico	15
Consulta 3: Servicios más consumidos por tipo de habitación.	15
Consulta 4: Tasa de ocupación por tipo de habitación.	16
Consulta 5: Ingresos totales por mes en el año actual en curso.	17
Consulta 6: Búsqueda textual de huéspedes por correo o nombre.	17
Consulta 7: Reservas próximas usando funciones de fecha.	18
Programación en la Base de datos	19
Función: noches reserva	19
Trigger: Validar reserva	21
Procedimiento almacenado: realizar check-out.....	24

Introducción

El sistema de gestión de reservas de hotel ha sido implementado en PostgreSQL 18 y está diseñado para administrar el ciclo completo de reservas, estancias(check-in/out), consumos de servicios y facturación de un hotel funcional.

Las reglas de negocios utilizadas fueron las siguientes:

- Un huésped puede realizar múltiples reservaciones.
- Una reservación cubre una habitación por un rango de fechas.
- Durante la estancia, el huésped puede consumir servicios adicionales (restaurant, lavandería, spa).
- Al hacer check-out se genera una factura consolidada.
- Una habitación no puede estar reservada dos veces en el mismo período.

La base de datos consta de 8 tablas principales y 3 objetos de programación en los que se incluye función, triggers y procedimiento. Se utilizaron las consultas sugeridas:

- Habitaciones disponibles en un rango de fechas dado.
- Huéspedes con mayor gasto histórico.
- Servicios más consumidos por tipo de habitación.
- Tasa de ocupación mensual por tipo de habitación.
- Ingresos totales por mes en el año en curso.

También se utilizó el procedimiento sugerido:

- Realizar el proceso de check-out: calcular el total a cobrar (habitación + servicios) y generar la factura correspondiente.

Y por último se tomó en cuenta el trigger que se nos sugirió:

- Al intentar insertar una reservación, verificar que la habitación no esté ocupada en ese período; si hay conflicto, lanzar un error.

Diagramas

Diagrama Entidad-relación (ER)

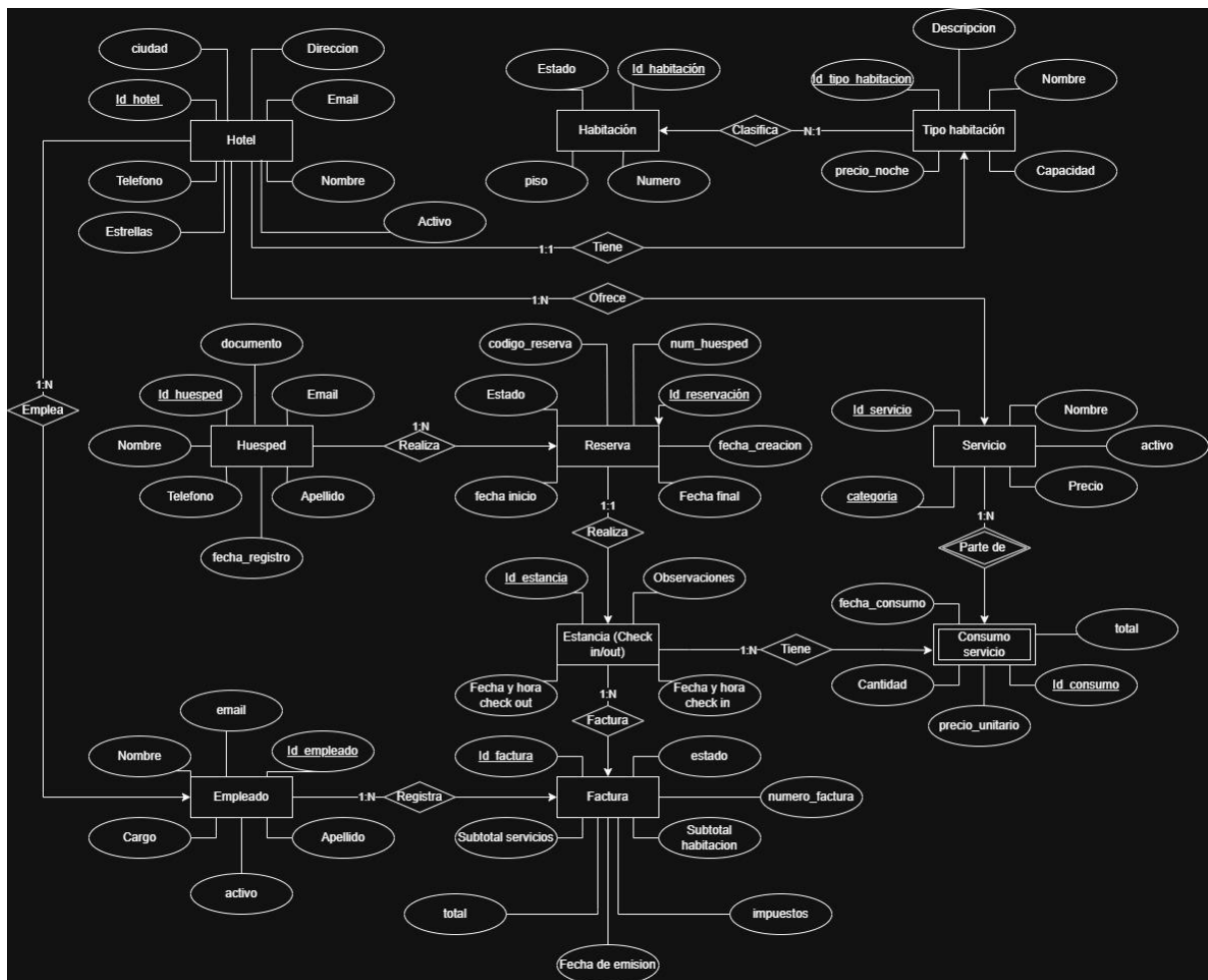
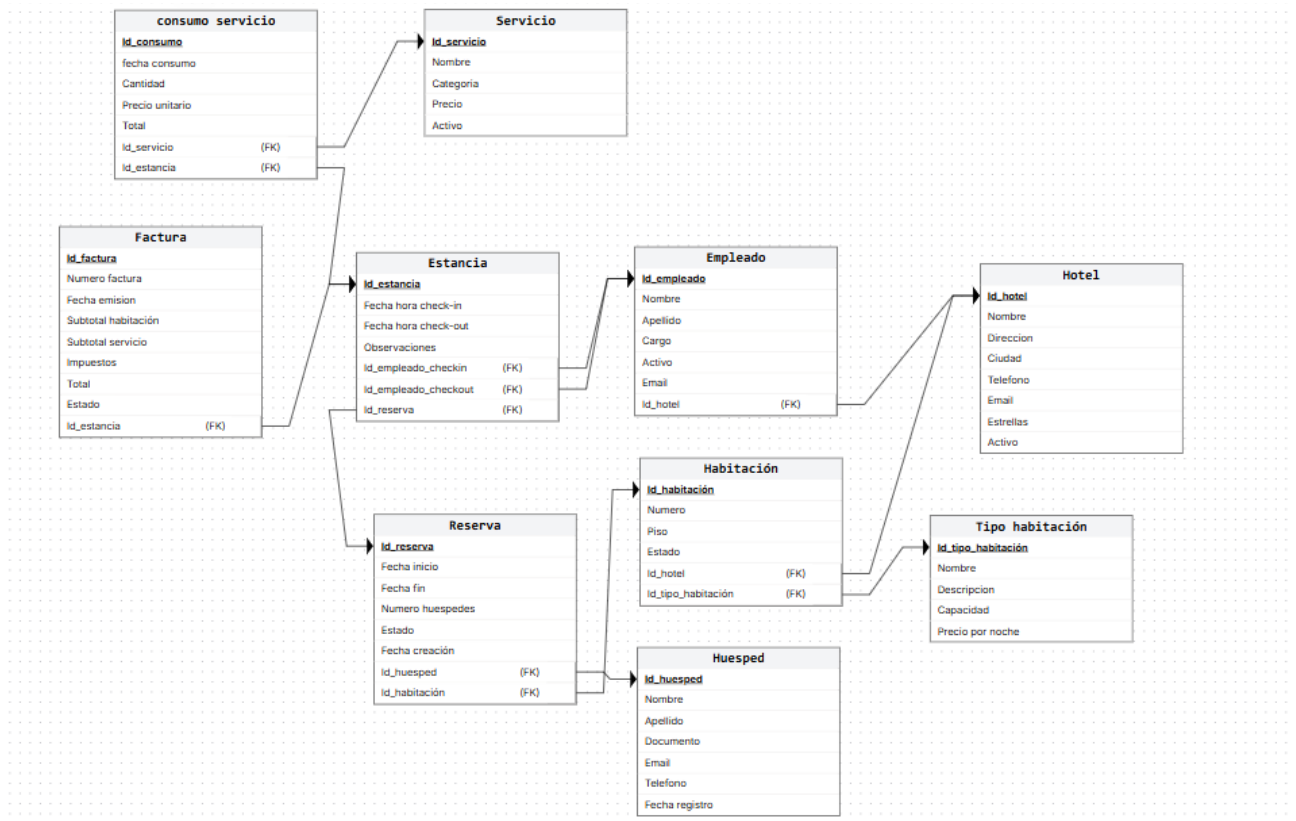


Diagrama Relacional normalizado (3FN)



Diccionario de datos

Tabla: hotel

Columna	Tipo	Llave/ nulo	Restricciones	Descripción
Id_hotel	Bigserial	Primary key	Identity	Identificador único del hotel, se genera siempre
Nombre	Varchar	No nulo	-	Nombre del hotel, no puede estar vacío
Dirección	Varchar	No nulo	-	Dirección del hotel, no puede quedar vacío
Ciudad	Varchar	No nulo	-	Ciudad en la que se encuentra el hotel, no puede quedar vacío
Telefono	Varchar	No nulo	-	Número de teléfono del hotel, no puede quedar vacío
Email	Varchar	No nulo	Unique	Email del hotel, tiene que ser único
Estrellas	Smallint	No nulo	Check (1-5)	Calidad del hotel, solo puede ser del 1-5
Activo	Boolean	No nulo	DEFAULT TRUE	Estado del hotel, si no se especifica será TRUE

Tabla: Tipo de habitación

Columna	Tipo	Llave/ nulo	Restricciones	Descripción
Id_tipo_habitación	Bigserial	Primary	Identity	Identificador único del tipo habitación, generado automáticamente
Nombre	Varchar	Not null	Unique	Nombre del tipo de habitación, tiene que ser único
Descripción	Text	Not null	-	Descripción de la habitación, no debe quedar vacío
Capacidad	Smallint	Not null	Check (>0)	Numero de huéspedes permitidos en la habitación, mayor a cero y no puede quedar vacío
Precio_noche	Numeric(10,2)	Not null	Check (>0)	El precio de cada habitación, mayor a cero y con dos decimales, no debe quedar vacío

Tabla: Habitación

Columna	Tipo	Llave/ nulo	Restricciones	Descripción
Id_habitación	Bigserial	Primary key	Identity	Identificador único de la habitación, generado automáticamente
Id_hotel	Bigint	Foreign key	ON UPDATE CASCADE ON DELETE RESTRICT,	Referencia al hotel que pertenece
Id_tipo_habitación	Bigint	Foreign key	ON UPDATE CASCADE ON DELETE RESTRICT,	Referencia al tipo de habitación que pertenece
Numero	Varchar	Not null		Numero identificador de la habitación
Piso	Smallint	Not null	Check (>0)	Numero de piso donde se ubica la habitación
Estado	Varchar	Not null	Default 'Disponible' Check (DISPONIBLE, MANTENIMIENTO, OCUPADA)	Estado actual de la habitación, estados permitidos: Disponible, Mantenimiento, Ocupada. Si se deja en blanco el estado se deja en Disponible
Constraint	-	-	uq_habitacion_hotel_numero	Restricción unique de id hotel, numero

Tabla: Huésped

Columna	Tipo	Llave/ nulo	Restricciones	Descripción
Id_huesped	Bigserial	Primary key	Identity	Identificador único del huésped, generado automáticamente
Nombres	Varchar	Not null	-	Nombres del huésped, no puede quedar vacío
Apellidos	Varchar	Not null	-	Apellidos del huésped, no puede quedar vacío
Documento	Varchar	Not null	Unique	Documento (dui) del huésped, debe ser único y no puede quedar vacío
Email	Varchar	Not null	Unique	Email del huésped, debe ser único y no puede quedar vacío
Telefono	Varchar	Not null	-	Telefono del huésped, no puede quedar vacío
Fecha_registro	Date	Not null	Default 'Current date'	Fecha de registro, no puede quedar vacío, sino se pone la fecha actual

Tabla: Empleado

Columna	Tipo	Llave/ nulo	Restricciones	Descripción
Id_empleado	Bigserial	Primary key	Identity	Identificador único de empleado, generado automáticamente
Id_hotel	Bigint	Foreign key	ON UPDATE CASCADE ON DELETE RESTRICT,	Referencia al hotel que pertenece
Nombres	Varchar	Not null	-	Nombres del empleado, no puede quedar vacío
Apellidos	Varchar	Not null	-	Apellidos del empleado, no puede quedar vacío
Cargo	Varchar	Not null	Check ('RECEPCION', 'ADMINISTRACION', 'LIMPIEZA', 'GERENCIA')	Cargo del empleado, permite 'RECEPCION', 'ADMINISTRACION', 'LIMPIEZA', 'GERENCIA'. No puede quedar vacío
Email	Varchar	Not null	Unique	Email del empleado, debe ser único y no puede quedar vacío
Activo	Boolean	Not null	Default 'True'	Contrato del empleado, no puede quedar vacío

Tabla: Reserva

Columna	Tipo	Llave/ nulo	Restricciones	Descripción
Id_reserva	Bigserial	Primary key	Identity	Identificador de reserva, se genera automaticamente
Codigo_reserva	Varchar	Not null	Unique	Codigo unico de reserva, debe ser unico y no puede quedar vacío
Id_huesped	Bigint	Foreing key	ON UPDATE CASCADE ON DELETE RESTRICT,	Referencia al huésped que realiza la reserva
Id_habitación	Bigint	Foreing key	ON UPDATE CASCADE ON DELETE RESTRICT,	Referencia a la habitación reservada
Fecha_inicio	Date	Not null	-	Fecha inicio de la estancia (check-in) y es obligatorio
Fecha_fin	Date	Not null	-	Fecha final de la estancia (check-out) y es obligatorio
Num_huesped	Smallint	Not null	Check (num_huésped > 0)	Número de personas que ocuparan la habitación, mayor a cero y es obligatorio
Estado	Varchar	Not null	Check ('CONFIRMADA', 'CHECK_IN', 'FINALIZADA', 'CANCELADA')	Estado actual de la reserva, predeterminado a 'CONFIRMADA'
Fecha_creación	Timestamp	Not null	Default 'Current timestamp'	Fecha y hora que se creó la reservación, por defecto la fecha y hora actual, obligatorio
Constraint	-	-	ck_reserva_fechas CHECK (fecha_fin > fecha_inicio)	Restricción check: Fecha_fin mayor que Fecha_inicio

Tabla: Estancia

Columna	Tipo	Llave/ nulo	Restricciones	Descripción
Id_estancia	Bigserial	Primary key	Identity	Identificador de estancia, se general automaticamente
Id_reserva	Bigint	Foreing key	Unique, ON UPDATE CASCADE ON DELETE RESTRICT	Referencia a la reserva de la estancia
Id_empleado_checkin	Bigint	Foreing key	ON UPDATE CASCADE ON DELETE RESTRICT	Referencia al empleado que realiza el check in de la estancia
Id_empleado_checkout	Bigint	Foreing key	ON UPDATE CASCADE ON DELETE RESTRICT	Referencia al empleado que realiza el check out de la estancia
fecha_hora_checkin	Timestamp	Not null	-	Fecha y hora del check in, obligatoria
fecha_hora_checkout	Timestamp	-	-	Fecha y hora del check out, obligatoria
Observaciones	Text	-	-	Observaciones de la estancia
Constraint	-	-	ck_estancia_checkout CHECK (fecha_hora_checkout IS NULL OR fecha_hora_checkout > fecha_hora_checkin)	Restricción la fecha y hora de checkout debe ser despues a la fecha y hora de checkin o puede ser NULL

Tabla: Servicio

Columna	Tipo	Llave/ nulo	Restricciones	Descripción
Id_servicio	Bigserial	Primary key	Identity	Identificador de servicio, generado automáticamente
Nombre	Varchar	Not null	Unique	Nombre del servicio, debe ser único y obligatorio
Categoria	Varchar	Not null	Check ('RESTAURANTE', 'LAVANDERIA', 'SPA', 'TRANSPORTE', 'OTRO')	Categoría al que pertenece el servicio, obligatorio
Precio	Numeric(10, 2)	Not null	Check (>=0)	Precio de servicio, debe ser mayor a cero y debe contener dos numerales, obligatorio
Activo	Boolean	Not null	Default 'True'	Estado del servicio, obligatorio, por defecto True

Tabla: Consumo servicio

Columna	Tipo	Llave/ nulo	Restricciones	Descripción
Id_consumo	Bigserial	Primary key	Identity	Identificador de consumo, se genera automáticamente
Id_estancia	Bigint	Foreign key	ON UPDATE CASCADE ON DELETE CASCADE	Referencia a la estancia donde se realizó el consumo, obligatorio
Id_servicio	Bigint	Foreign key	ON UPDATE CASCADE ON DELETE CASCADE	Referencia al servicio consumido, obligatorio
Fecha_consumo	Timestamp	Not null	Default Current_timestamp	Fecha y hora de consumo, por defecto fecha y hora actual
Cantidad	Smallint	Not null	CHECK (cantidad > 0)	Cantidad del servicio consumido, mayor a cero
Precio_unitario	Numeric(10, 2)	Not null	CHECK (precio_unitario >= 0)	Precio unitario del servicio, mayor a cero
Total	Numeric(10, 2)	-	GENERATED ALWAYS AS (cantidad * precio_unitario) STORED	Total, calculado automáticamente: cantidad × precio_unitario. No se puede modificar directamente.

Tabla: Factura

Columna	Tipo	Llave/ nulo	Restricciones	Descripción
Id_factura	Bigserial	Primary key	Identity	Identificador de factura, se genera automáticamente
Id_estancia	Bigint	Foreing key	ON UPDATE CASCADE ON DELETE CASCADE	Referencia a la estancia de factura, obligatorio
Numero_factura	Varchar	Not null	Unique	Codigo de numero de facura, unico y obligatorio
Fecha_emision	Timestamp	Not null	Default current_timestamp	Fecha de emision de factura, por defecto fecha y hora actual, obligatorio
Subtotal_habitación	Numeric(10, 2)	Not null	(subtotal_habitacion >= 0)	Costo de la habitación (noches × precio_noche). Debe ser >= 0.
Subtotal_servicio	Numeric(10, 2)	Not null	(subtotal_servicios >= 0)	Costo total de servicios consumidos. Debe ser >= 0.
Impuesto	Numeric(10, 2)	Not null	(impuestos >= 0)	Impuestos aplicados (13% IVA). Debe ser >= 0.
Total	Numeric(10, 2)	Not null	(total >= 0)	Total a pagar (subtotal_habitacion + subtotal_servicios + impuestos). Debe ser >= 0.
Estado	Varchar	Not null	DEFAULT 'EMITIDA' CHECK (estado IN ('EMITIDA', 'PAGADA', 'ANULADA')	Estado de la factura. Valores: EMITIDA, PAGADA, ANULADA. Por defecto EMITIDA

Consultas SQL en el Sistema de reservas

Consulta 1: Habitaciones disponibles en un rango de fechas

Muestra todas las habitaciones disponibles para reservar en un rango de fechas específico, considerando cuales habitaciones no se encuentran disponibles debido a estar ocupadas o en mantenimiento.

```
SELECT
    h.id_habitacion,
    ho.nombre AS hotel,
    h.numero,
    th.nombre AS tipo_habitacion,
    th.capacidad,
    th.precio_noche
FROM habitacion h
JOIN hotel ho ON ho.id_hotel = h.id_hotel
JOIN tipo_habitacion th ON th.id_tipo_habitacion = h.id_tipo_habitacion
WHERE h.estado = 'DISPONIBLE'
    AND NOT EXISTS (
        SELECT 1
        FROM reserva r
        WHERE r.id_habitacion = h.id_habitacion
            AND r.estado IN ('CONFIRMADA', 'CHECK_IN')
            AND daterange(r.fecha_inicio, r.fecha_fin, '[]')
                && daterange(DATE '2026-01-10', DATE '2026-01-14', '[]')
    )
ORDER BY ho.nombre, h.numero;
```


	123 id_habitacion	AZ hotel	AZ numero	AZ tipo_habitacion	123 capacidad	123 precio_noche
1	1	Hotel Brisas del Pacifico	101	Individual	1	55
2	2	Hotel Brisas del Pacifico	102	Individual	1	55
3	3	Hotel Brisas del Pacifico	103	Individual	1	55
4	4	Hotel Brisas del Pacifico	104	Doble	2	85
5	5	Hotel Brisas del Pacifico	105	Doble	2	85
6	6	Hotel Brisas del Pacifico	106	Doble	2	85
7	7	Hotel Brisas del Pacifico	107	Familiar	4	130
8	8	Hotel Brisas del Pacifico	108	Familiar	4	130
9	9	Hotel Brisas del Pacifico	109	Suite	3	210
10	10	Hotel Brisas del Pacifico	110	Suite	3	210
11	11	Hotel Brisas del Pacifico	111	Individual	1	55
12	12	Hotel Brisas del Pacifico	112	Individual	1	55
13	13	Hotel Brisas del Pacifico	201	Individual	1	55
14	14	Hotel Brisas del Pacifico	202	Doble	2	85
15	15	Hotel Brisas del Pacifico	203	Doble	2	85

Consulta 2: Huéspedes con mayor gasto histórico

Identifica los 10 huéspedes que más han gastado en el hotel, sumando todas sus facturas históricas.

SELECT

```

    hu.id_huesped,
    hu.nombres || ' ' || hu.apellidos AS huesped,
    COUNT(f.id_factura) AS facturas_emitidas,
    SUM(f.total) AS gasto_total

```

FROM huesped hu

JOIN reserva r ON r.id_huesped = hu.id_huesped

JOIN estancia e ON e.id_reserva = r.id_reserva

JOIN factura f ON f.id_estancia = e.id_estancia

GROUP BY hu.id_huesped, hu.nombres, hu.apellidos

ORDER BY gasto_total DESC

LIMIT 10;

	123 id_huesped	A-Z huesped	123 facturas_emitidas	123 gasto_total
1	20	Huesped20 Apellido20	1	1.073,5
2	29	Huesped29 Apellido29	1	824,9
3	10	Huesped10 Apellido10	1	734,5
4	8	Huesped8 Apellido8	1	718,68
5	40	Huesped40 Apellido40	1	717,55
6	17	Huesped17 Apellido17	1	668,96
7	38	Huesped38 Apellido38	1	641,84
8	5	Huesped5 Apellido5	1	632,8
9	19	Huesped19 Apellido19	1	610,2
10	35	Huesped35 Apellido35	1	610,2

Consulta 3: Servicios más consumidos por tipo de habitación.

Analiza los servicios más consumidos en base al tipo de habitación, mostrando su cantidad y total generado de manera histórica

SELECT

```

    th.nombre AS tipo_habitacion,
    s.categoria,
    s.nombre AS servicio,
    SUM(cs.cantidad) AS cantidad_consumida,
    SUM(cs.total) AS total_generado

```

FROM consumo_servicio cs

JOIN servicio s ON s.id_servicio = cs.id_servicio

JOIN estancia e ON e.id_estancia = cs.id_estancia

JOIN reserva r ON r.id_reserva = e.id_reserva

JOIN habitacion h ON h.id_habitacion = r.id_habitacion

JOIN tipo_habitacion th ON th.id_tipo_habitacion = h.id_tipo_habitacion

GROUP BY th.nombre, s.categoria, s.nombre

ORDER BY th.nombre, cantidad_consumida DESC;

	AZ tipo_habitacion	AZ categoria	AZ servicio	123 cantidad_consumida	123 total_generado
1	Doble	SPA	Masaje relajante	34	1.530
2	Doble	OTRO	Minibar	24	360
3	Doble	LAVANDERIA	Lavado express	23	230
4	Doble	SPA	Piedras spa	20	1.200
5	Doble	RESTAURANTE	Desayuno buffet	20	240
6	Doble	OTRO	Decoracion especial	16	560
7	Familiar	TRANSPORTE	Traslado aeropuerto	30	900
8	Familiar	LAVANDERIA	Lavado normal	22	143
9	Familiar	TRANSPORTE	Tour local	20	1.100
10	Familiar	RESTAURANTE	Cena ejecutiva	15	330

Consulta 4: Tasa de ocupación por tipo de habitación.

Muestra la tasa de ocupación para cada tipo de habitación, mostrando cuantas habitaciones de ese tipo existen y cuantas reservas validas han sido realizadas.

SELECT

```

th.nombre AS tipo_habitacion,
COUNT(DISTINCT h.id_habitacion) AS total_habitaciones,
COUNT(DISTINCT r.id_reserva) FILTER (
    WHERE r.estado IN ('CONFIRMADA', 'CHECK_IN', 'FINALIZADA')
) AS reservas_validas,
ROUND(
    COUNT(DISTINCT r.id_reserva) FILTER (
        WHERE r.estado IN ('CONFIRMADA', 'CHECK_IN', 'FINALIZADA')
    )::NUMERIC / NULLIF(COUNT(DISTINCT h.id_habitacion), 0),
    2
) AS reservas_por_habitacion

```

FROM tipo_habitacion th

JOIN habitacion h ON h.id_tipo_habitacion = th.id_tipo_habitacion

LEFT JOIN reserva r ON r.id_habitacion = h.id_habitacion

GROUP BY th.nombre

ORDER BY reservas_por_habitacion DESC;

	AZ tipo_habitacion	123 total_habitaciones	123 reservas_validas	123 reservas_por_habitacion
1	Doble	18	33	1,83
2	Familiar	12	22	1,83
3	Individual	18	33	1,83
4	Suite	12	22	1,83

Consulta 5: Ingresos totales por mes en el año actual en curso.

Genera un reporte financiero que muestra los ingresos totales de un mes, desglosando los ingresos ganados por habitación, servicios e impuestos.

SELECT

```
    TO_CHAR(f.fecha_emision, 'YYYY-MM') AS mes,  
    COUNT(f.id_factura) AS total_facturas,  
    SUM(f.subtotal_habitacion) AS ingresos_habitacion,  
    SUM(f.subtotal_servicios) AS ingresos_servicios,  
    SUM(f.impuestos) AS impuestos,  
    SUM(f.total) AS ingresos_totales
```

FROM factura f

GROUP BY TO_CHAR(f.fecha_emision, 'YYYY-MM')

ORDER BY mes;

	A2 mes	123 total_facturas	123 ingresos_habitacion	123 ingresos_servicios	123 impuestos	123 ingresos_totales
1	2026-06	45	9.390	8.164	2.282,02	19.836,02

Consulta 6: Búsqueda textual de huéspedes por correo o nombre.

Realiza una búsqueda rápida de los huéspedes en base a su correo electrónico o nombre.

SELECT

```
    id_huesped,  
    UPPER(nombres || ' ' || apellidos) AS huesped,  
    email,  
    telefono
```

FROM huesped

WHERE LOWER(email) LIKE '%huesped1%'

```
    OR LOWER(nombres || ' ' || apellidos) LIKE '%huesped1%'
```

ORDER BY huesped;

	123 id_huesped	AZ huesped	AZ email	AZ telefono
1	100	HUESPED100 APELLIDO100	huesped100@correo.com	7000-0100
2	101	HUESPED101 APELLIDO101	huesped101@correo.com	7000-0101
3	102	HUESPED102 APELLIDO102	huesped102@correo.com	7000-0102
4	103	HUESPED103 APELLIDO103	huesped103@correo.com	7000-0103
5	104	HUESPED104 APELLIDO104	huesped104@correo.com	7000-0104
6	105	HUESPED105 APELLIDO105	huesped105@correo.com	7000-0105
7	106	HUESPED106 APELLIDO106	huesped106@correo.com	7000-0106
8	107	HUESPED107 APELLIDO107	huesped107@correo.com	7000-0107
9	108	HUESPED108 APELLIDO108	huesped108@correo.com	7000-0108
10	109	HUESPED109 APELLIDO109	huesped109@correo.com	7000-0109

Consulta 7: Reservas próximas usando funciones de fecha.

Muestra las habitaciones reservadas, los huéspedes que las reservaron, la habitación y la fecha de inicio y final.

SELECT

```

    r.codigo_reserva,
    hu.nombres || ' ' || hu.apellidos AS huesped,
    h.numero AS habitacion,
    r.fecha_inicio,
    r.fecha_fin,
    r.fecha_inicio - CURRENT_DATE AS dias_para_llegada

```

FROM reserva r

JOIN huesped hu ON hu.id_huesped = r.id_huesped

JOIN habitacion h ON h.id_habitacion = r.id_habitacion

WHERE r.estado = 'CONFIRMADA'

ORDER BY r.fecha_inicio

LIMIT 15;

	AZ codigo_reserva	AZ huesped	AZ habitacion	fecha_inicio	fecha_fin	123 dias_para_llegada
1	RES-000081	Huesped81 Apellido81	209	2026-01-25	2026-01-26	-145
2	RES-000091	Huesped91 Apellido91	307	2026-01-25	2026-01-27	-145
3	RES-000096	Huesped96 Apellido96	312	2026-01-25	2026-01-26	-145
4	RES-000106	Huesped106 Apellido106	410	2026-01-25	2026-01-27	-145
5	RES-000086	Huesped86 Apellido86	302	2026-01-25	2026-01-28	-145
6	RES-000101	Huesped101 Apellido101	405	2026-01-25	2026-01-28	-145
7	RES-000076	Huesped76 Apellido76	204	2026-01-25	2026-01-27	-145
8	RES-000097	Huesped97 Apellido97	401	2026-01-26	2026-01-28	-144
9	RES-000077	Huesped77 Apellido77	205	2026-01-26	2026-01-29	-144
10	RES-000107	Huesped107 Apellido107	411	2026-01-26	2026-01-29	-144

Programación en la Base de datos

Función: noches reserva

Esta función se encarga de calcular el número de noches que durara una reserva mediante el cálculo de la diferencia entre fecha de inicio y fecha de fin.

```
CREATE OR REPLACE FUNCTION fn_noches_reserva(p_id_reserva BIGINT)
RETURNS INTEGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_noches INTEGER;
BEGIN
    SELECT (fecha_fin - fecha_inicio)
    INTO v_noches
    FROM reserva
    WHERE id_reserva = p_id_reserva;

    RETURN COALESCE(v_noches, 0);
END;
$$;
```

La demostración de la función se pone en funcionamiento mediante:

```
SELECT
    id_reserva,
    codigo_reserva,
    fn_noches_reserva(id_reserva) AS noches
FROM reserva
ORDER BY id_reserva
LIMIT 5;
```

Y se vería de tal manera que como se muestra en la imagen anexa.

123 id_reserva ▼	A-Z codigo_reserva ▼	123 noches ▼
1	RES-000001	2
2	RES-000002	3
3	RES-000003	1
4	RES-000004	2
5	RES-000005	3

Trigger: Validar reserva

Se encarga de validar que una reserva sea válida antes de insertarla o actualizarla en la base de datos mediante el uso de la función validar reserva.

```
CREATE OR REPLACE FUNCTION fn_validar_reserva()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_capacidad SMALLINT;
BEGIN
    SELECT th.capacidad
    INTO v_capacidad
    FROM habitacion h
    JOIN tipo_habitacion th ON th.id_tipo_habitacion =
h.id_tipo_habitacion
    WHERE h.id_habitacion = NEW.id_habitacion;
    IF NEW.num_huespedes > v_capacidad THEN
        RAISE EXCEPTION 'La habitacion solo permite % huesped(es), pero la
reserva incluye %.',
            v_capacidad, NEW.num_huespedes;
    END IF;
    IF EXISTS (
        SELECT 1
        FROM reserva r
        WHERE r.id_habitacion = NEW.id_habitacion
            AND r.id_reserva <> COALESCE(NEW.id_reserva, -1)
            AND r.estado IN ('CONFIRMADA', 'CHECK_IN')
            AND daterange(r.fecha_inicio, r.fecha_fin, '[')
                && daterange(NEW.fecha_inicio, NEW.fecha_fin, '[')
    ) THEN
        RAISE EXCEPTION 'La habitacion ya esta reservada en ese rango de
fechas.';
```



```
        END IF;

        RETURN NEW;

END;

$;$
```

Después hace uso del Trigger para disparar automáticamente la validación de una reserva antes de que sea insertada o actualizada en la base de datos.

```
CREATE TRIGGER trg_validar_reserva

BEFORE INSERT OR UPDATE OF id_habitacion, fecha_inicio, fecha_fin,
num_huespedes, estado

ON reserva

FOR EACH ROW

WHEN (NEW.estado IN ('CONFIRMADA', 'CHECK_IN'))

EXECUTE FUNCTION fn_validar_reserva();
```

Para la demostración se hace uso de una reserva que se quiere realizar cuando una habitación ya está ocupada.

```
INSERT INTO reserva (

    codigo_reserva,

    id_huesped,

    id_habitacion,

    fecha_inicio,

    fecha_fin,

    num_huespedes,

    estado

)

VALUES (

    'RES-CONFLICTO',

    1,

    1,

    DATE '2026-01-05',
```

```
DATE '2026-01-07',  
1,  
'CONFIRMADA'  
);
```

Mostrándose un error que indica que la habitación ya está reservada para esas fechas como se puede apreciar en la imagen anexa.

```
SQL Error [P0001]: ERROR: La habitacion ya esta reservada  
en ese rango de fechas.  
Where: PL/pgsql function fn_validar_reserva() line 25  
at RAISE
```

Procedimiento almacenado: realizar check-out

Procedimiento almacenado que se utiliza para genera la factura y calcular impuesto de los servicios utilizados, a su vez actualizando el estado de la factura.

```
CREATE OR REPLACE PROCEDURE sp_realizar_checkout(
```

```

        p_id_reserva BIGINT,
        p_id_empleado_checkout BIGINT
    )
LANGUAGE plpgsql
AS $$
DECLARE
    v_id_estancia BIGINT;
    v_noches INTEGER;
    v_precio_noche NUMERIC(10,2);
    v_subtotal_habitacion NUMERIC(10,2);
    v_subtotal_servicios NUMERIC(10,2);
    v_impuestos NUMERIC(10,2);
    v_total NUMERIC(10,2);
BEGIN
    SELECT e.id_estancia, fn_noches_reserva(r.id_reserva), th.precio_noche
    INTO v_id_estancia, v_noches, v_precio_noche
    FROM estancia e
    JOIN reserva r ON r.id_reserva = e.id_reserva
    JOIN habitacion h ON h.id_habitacion = r.id_habitacion
    JOIN tipo_habitacion th ON th.id_tipo_habitacion =
h.id_tipo_habitacion
    WHERE r.id_reserva = p_id_reserva;

    IF v_id_estancia IS NULL THEN
        RAISE EXCEPTION 'No existe una estancia para la reserva %.',
p_id_reserva;
    END IF;

    v_subtotal_habitacion := v_noches * v_precio_noche;

    SELECT COALESCE(SUM(total), 0)

```

```

    INTO v_subtotal_servicios

    FROM consumo_servicio

    WHERE id_estancia = v_id_estancia;

    v_impuestos := ROUND((v_subtotal_habitacion + v_subtotal_servicios) *
0.13, 2);

    v_total := v_subtotal_habitacion + v_subtotal_servicios + v_impuestos;

    UPDATE estancia
    SET fecha_hora_checkout = CURRENT_TIMESTAMP,
        id_empleado_checkout = p_id_empleado_checkout
    WHERE id_estancia = v_id_estancia
        AND fecha_hora_checkout IS NULL;

    UPDATE reserva
    SET estado = 'FINALIZADA'
    WHERE id_reserva = p_id_reserva;

    INSERT INTO factura (
        id_estancia,
        numero_factura,
        subtotal_habitacion,
        subtotal_servicios,
        impuestos,
        total,
        estado
    )
    VALUES (
        v_id_estancia,
        'FAC-' || LPAD(v_id_estancia::TEXT, 6, '0'),
        v_subtotal_habitacion,

```

```

        v_subtotal_servicios,
        v_impuestos,
        v_total,
        'EMITIDA'
    )
    ON CONFLICT (id_estancia) DO NOTHING;
END;
$$;

```

Para la demostración se crea una factura para una estancia la cual aún no tenga factura mediante:

```

DO $$
DECLARE
    v_reserva BIGINT;
BEGIN
    SELECT r.id_reserva
    INTO v_reserva
    FROM reserva r
    JOIN estancia e ON e.id_reserva = r.id_reserva
    WHERE r.estado = 'CHECK_IN'
    AND NOT EXISTS (
        SELECT 1
        FROM factura f
        WHERE f.id_estancia = e.id_estancia
    )
    ORDER BY r.id_reserva
    LIMIT 1;

    IF v_reserva IS NOT NULL THEN
        CALL sp_realizar_checkout(v_reserva, 1);
    END IF;

```

```
END $$;
```

Después se verifica la creación de la factura generada por el procedimiento mediante el uso de una consulta rápida:

```
SELECT
    f.numero_factura,
    r.codigo_reserva,
    f.subtotal_habitacion,
    f.subtotal_servicios,
    f.impuestos,
    f.total,
    f.estado
FROM factura f
JOIN estancia e ON e.id_estancia = f.id_estancia
JOIN reserva r ON r.id_reserva = e.id_reserva
ORDER BY f.id_factura DESC
LIMIT 5;
```

Por lo que se mostraría como se ve en la imagen anexa.

	AZ numero_factura	AZ codigo_reserva	123 subtotal_habitacion	123 subtotal_servicios	123 impuestos	123 total	AZ estado
1	FAC-000046	RES-000046	170	204	48,62	422,62	EMITIDA
2	FAC-000045	RES-000045	85	125	27,3	237,3	PAGADA
3	FAC-000044	RES-000044	255	70	42,25	367,25	EMITIDA
4	FAC-000043	RES-000043	110	191	39,13	340,13	EMITIDA
5	FAC-000042	RES-000042	55	126	23,53	204,53	PAGADA